

IS6200 Blockchain and Smart Contract Development

Group Project Report

Blockchain-Based Luxury Supply Chain Traceability System

Group Number: 30

Members:

Leijie LIU, Shuaihui MA, Zechun WANG, Shi YE, Junlong ZENG

Date: April 4, 2026

Table of Contents

Abstract.....	2
1. Problem & Solution	3
1.1 Problem Identification	3
1.2 Proposed Solution.....	4
2. System Design and Architecture.....	6
2.1 System Overall Architecture.....	6
2.2 Use Case Diagrams.....	6
2.3 Class Diagram.....	8
2.4 Product Lifecycle State Diagram.....	9
2.5 Activity Diagram	10
2.6 Use Case Descriptions	10
3. Market & Partners.....	16
3.1 Market Analysis.....	16
3.2 Potential Partners	17
4. Competition & Risks.....	19
4.1 Competitive Analysis.....	19
5. Value & Distribution.....	21
5.1 Value Proposition	21
5.2 Business Model and Distribution.....	22
6. Conclusion	24
7. References.....	25
Appendix I: Group Project source code.....	27
supply_chain_smart.sol:	27
NFT.sol:.....	37
Appendix II: Group Project Test screenshot.....	44

Abstract

As one of the most distinctive industries in the world, the luxury goods industry has the characteristics of high individual product value and strong brand premium. However, due to the numerous links involved in the process from manufacturing to sales, which require passing through brand factories, international logistics, bonded warehousing, domestic transportation, brand authorized stores, and other links, it is extremely easy to encounter fraud problems, and consumers also need to face the extremely difficult problem of verifying authenticity channels. Therefore, in order to solve the problems in the luxury goods supply chain, this project adopts smart contracts and non fungible token (NFT) technology to create a blockchain based luxury goods tracking system to track each luxury item.

1. Problem & Solution

1.1 Problem Identification

There are a lot of individuals that work in the luxury goods supply chain to get a product from the manufacturing to the retailer. The brand, the manufacturer, the logistics supplier, and the retailer are all individuals that work together. The standard supply chain, on the other hand, has a lot of trouble handling information and building trust. These issues become most clear when it comes to changing ownership and handling deals.

1.1.1 The issue of ownership transfer

Usually, the change of ownership was written down on paper or kept in a central computer in the old method. This strategy makes it tougher to see things. Participants can only view the data in their own system; they can't see the complete lifespan of the product. This makes it challenging for both consumers and authorities to find out where the things truly originate from and how they get to the retailers.

Also, centralized storage makes it feasible for data to be lost or modified. If someone hacks into the database or someone within the organization does anything improper, the historical records might be modified, which would make the data less credible. When different firms don't have the same standards, it's also more probable that data will be inconsistent.

When there is a disagreement over who owns something or if it is real, all parties generally have to depend on a third party to handle it since there are no unified and accurate records. This will make the time and money costs go up.

When it comes to transferring ownership, typical supply chains have three basic problems: they are not very clear, the data is not very dependable, and they are not very good at resolving conflicts quickly.

1.1.2 Transaction processing mechanism issue

There are additional problems with the way transactions are processed, in addition to the question of ownership. Most of the time, the existing system employs centralized

platforms to keep track of transactions and change their status. When more than one person works together, this method is likely to have complications.

For one thing, third-party organizations are needed for transaction execution and verification. This makes trust more expensive. People who use the platform need to trust that it is safe and fair, yet it could not be. On the other hand, supply chain data is spread out among numerous systems, which means that the information is not connected.

Because there isn't a single way for everyone to share data, it's hard for everyone to have all the information they need on time, which lowers overall efficiency.

However, a lot of transactions still need to be done by hand or with some automation.

This is likely to cause errors and slow down the pace of processing. This problem becomes evident in supply chains that are hard to understand.

1.2 Proposed Solution

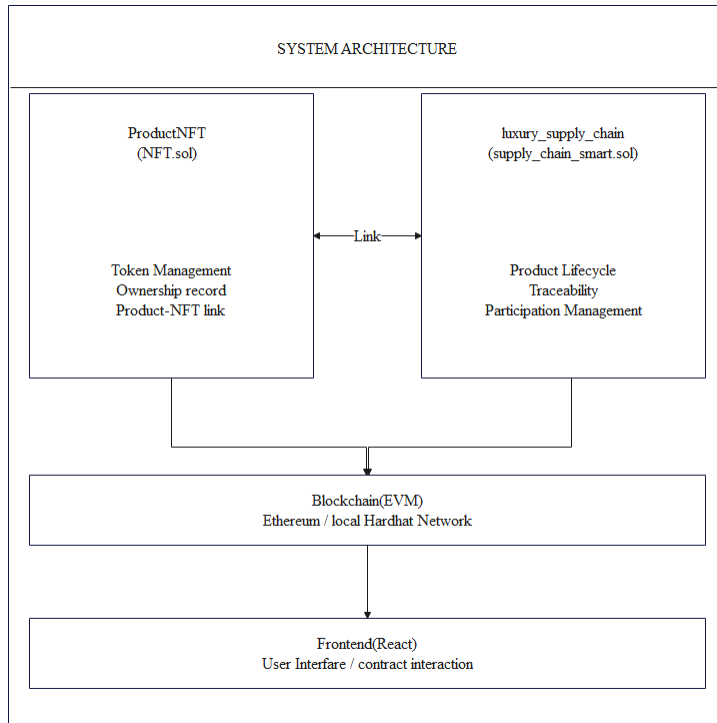
To fix these issues, this project made a blockchain-based system for keeping track of expensive things. Smart contracts and NFTs are used in the tech to make data more open and to automate tasks. Things that are built on blockchain, like cryptocurrency and NFTs, are at the center of this change. [11]. This system is based on blockchain. Smart contracts are responsible for executing the core logic. Smart contracts have sped up the use of blockchain technology in many areas by letting people involved in a transaction agree to something in code [10]. Meanwhile, each luxury item will correspond to an NFT, which serves as a unique digital identity. Through this design, the system can complete data recording and process execution without the need for intermediaries. Smart contracts are in charge of the major functions of the system. The system will first take care of the participants. Before brands, manufacturers, shipping companies, and retailers can use the data, they need to get permission. This may make the system safer. Then, when a product is made, it will get a unique ID and some basic information will be saved. This makes sure that the product is one of a kind in the system. Next, the system employs states to show the many stages of the product's life cycle, such when it is being made, when it is being sent, and when it is sold. The blockchain will keep track of every change in status, creating a full record. The system will also make an NFT for each product at the same time. This NFT is a one-of-a-kind ID and evidence that you own something. The smart

contract takes care of the transfer of ownership on its own, which might mean that less work has to be done by people. Blockchain technology changes the luxury supply chain by fixing its biggest problems with four linked processes. First of all, distributed ledgers make a "single source of truth," which helps do rid of information silos and makes sure that everyone can see the same clear facts. The fact that the ledger can't be changed makes it a secure audit trail. To be sure that a product is real and to keep fakes off the market, this is important. The open nature of the system also means we don't have to rely on expensive third-party brokers as much. These steps help us get around regulatory problems and lower the risks of doing business. Lastly, smart contracts go even further because they do important things automatically when certain conditions are met. For example, they move ownership or settle payments. Switching from doing things by hand to this kind of digital, code-based thinking not only speeds things up, but it also makes it much less likely that people will make mistakes. All of these things make the traditional supply chain into an environment that is much stronger, more open, and able to take care of itself.

2. System Design and Architecture

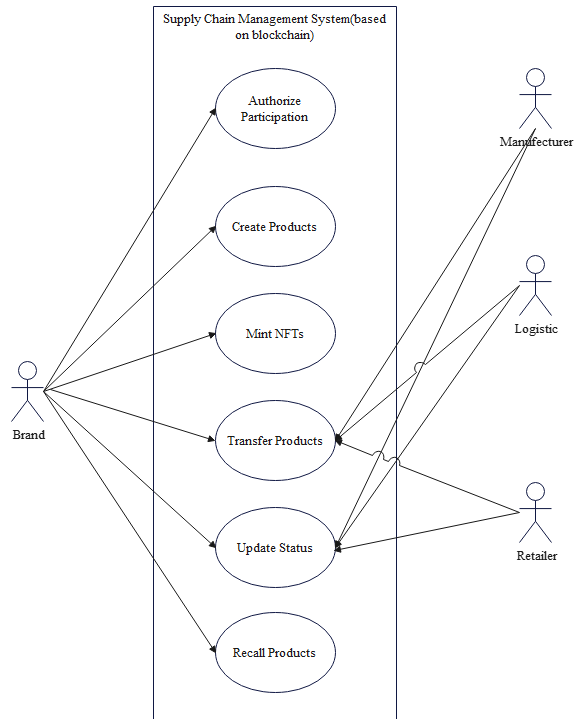
2.1 System Overall Architecture

The system is composed of two smart contracts which work together. This system gives a complete luxury product supply chain tracking solution with NFT based certification.

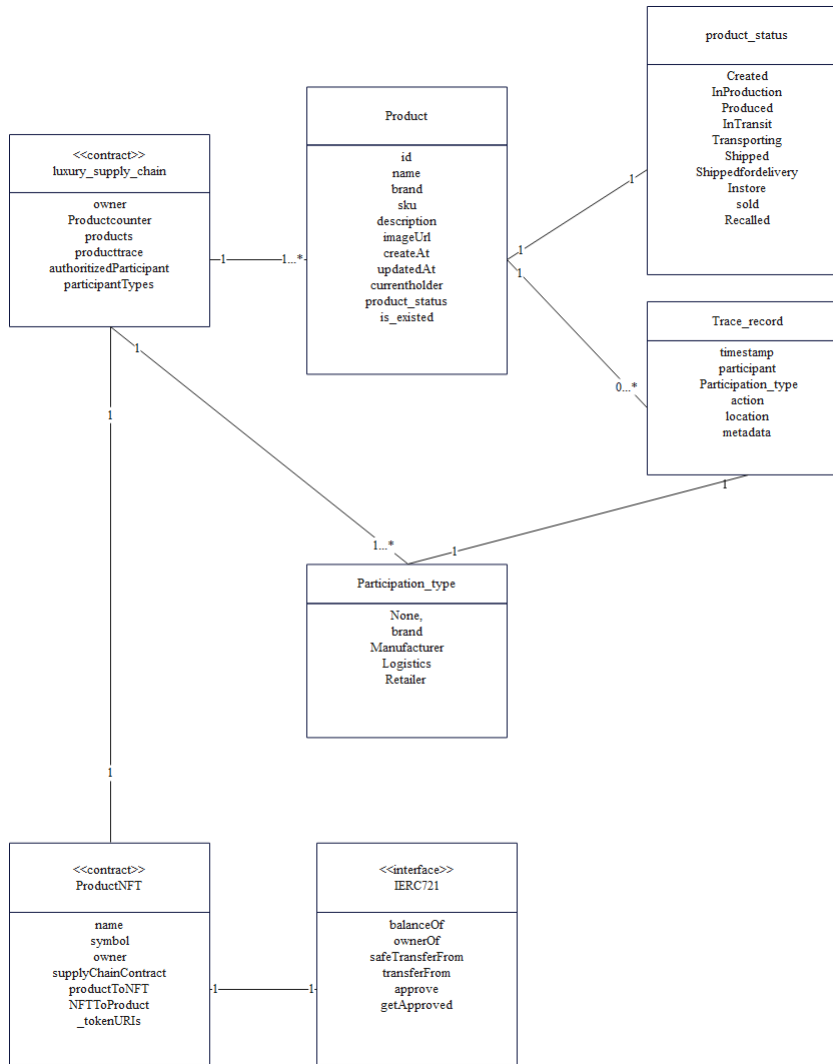


2.2 Use Case Diagrams

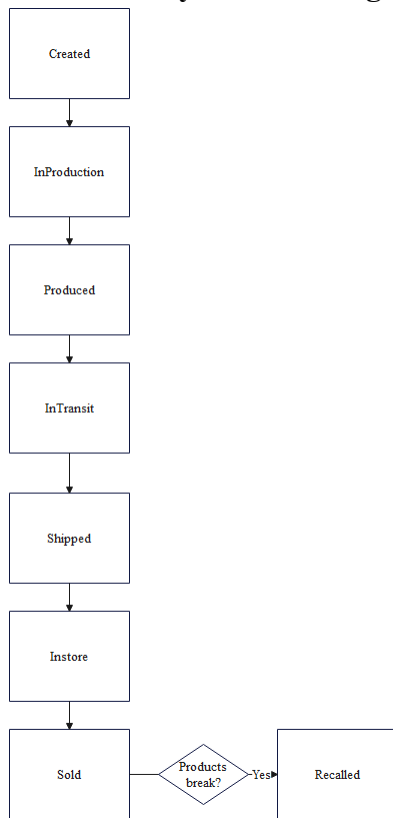
This Diagram also shows the permission of each participant. For example, Brand have the highest level permission of . On the other hand, the manufacturer, the logistics and the retailer can only update status and transfer the product .



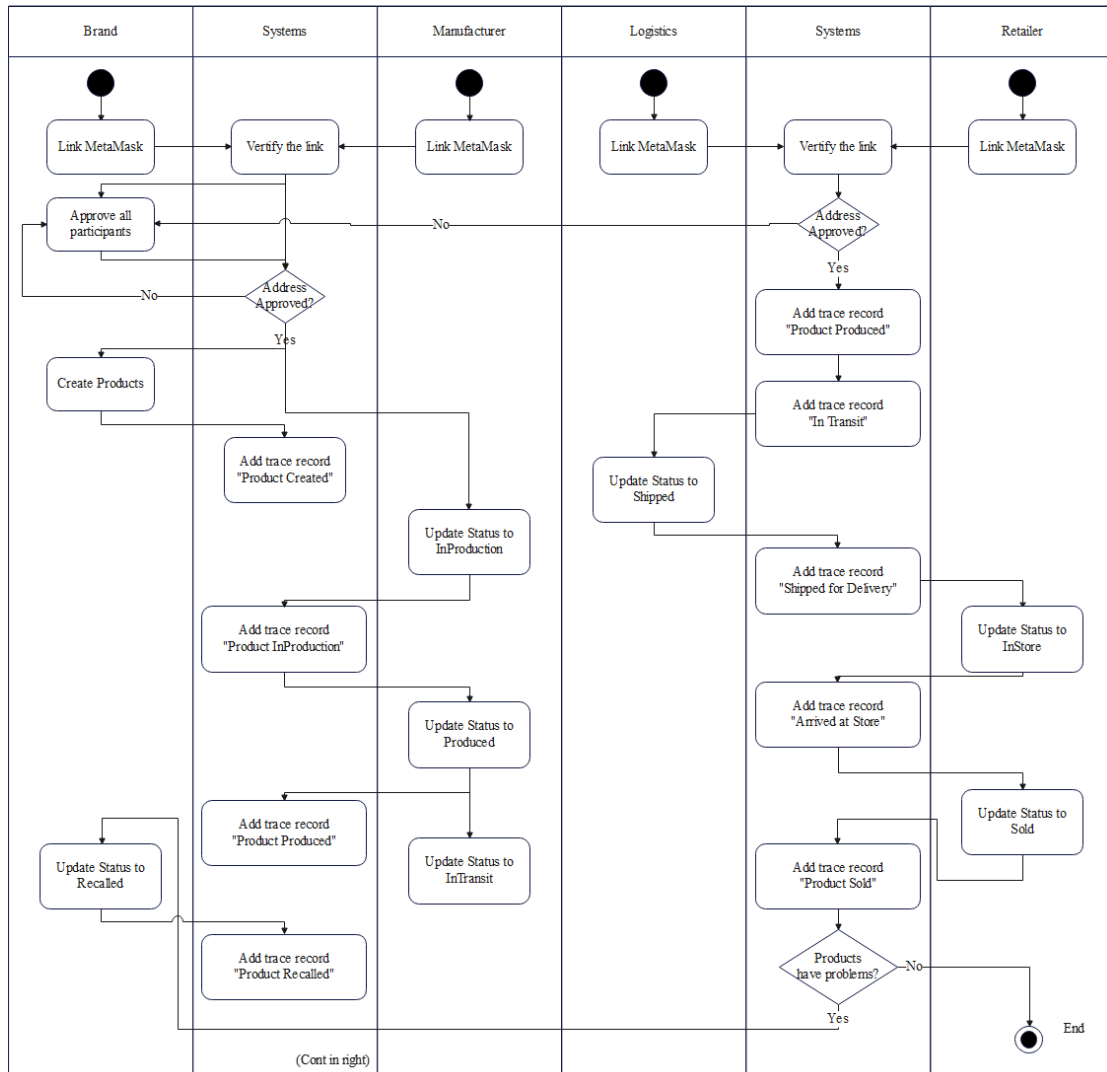
2.3 Class Diagram



2.4 Product Lifecycle State Diagram



2.5 Activity Diagram



This activity diagram show basic workflow for all six core functions.

2.6 Use Case Descriptions

Use Case 1. Authorize Participants

Flow of activities for scenario of Authorize Participants

Main Flow:

- 1.The Brand connects to the Website using their wallet.
- 2.The system verifies the wallet address is authorized as a Brand participant.
- 3.The Brand selects "Authorize Participants" button from the main menu.

4. The system displays the "Participant Authorization" interface.
5. The Brand enters the participant's wallet address to authorize.
6. The system checks if the address is already authorized.
7. The Brand selects the participant types: Manufacturer, Logistics, or Retailer.
8. The Brand clicks "Submit" to authorize the participant.
9. The system calls the `authorizeparticipant()` smart contract function.
10. The smart contract adds the address to authorized participants and sets the participant type.
11. The system emits the `ParticipantsAuthorized` event.
12. The system displays "Participant Authorized Successfully" with confirmation details.

Exception Conditions: 2. If the wallet address is not a Brand participant, the system will display Only Brand can authorize participants and return to step 1. 5. If the entered address is invalid (zero address), display Invalid Address and return to step 5. 6. If the address is already authorized, display Participant Already Authorized and return to step 5. 7. If the Brand try to authorize another Brand, display Cannot authorize Brand type and return to step 7. 9. If the blockchain transaction is not done, display Transaction Failed and return to step 8.

Use Case 2. Create Product

Flow of activities for scenario of Create Product

Main Flow:

1. After authorizing all participants, the Brand selects "Create Product" button from the main menu.
2. The system displays the "Create Product" form interface.
3. The Brand enters product details: name, brand, SKU, description, and image URL.
4. The Brand clicks "Submit" to create the product.

5. The system validates all required fields are filled.
6. The system calls the createproduct() smart contract function.
7. The smart contract generates a unique Product ID and stores the product information on blockchain.
8. The system automatically adds an initial trace record: "Product Created" at Brand Headquarters, showed in Product List column.
9. The system displays the message "Product Created Successfully" with the new Product ID.

Exception Conditions: 5. Is Empty or Invalid: if any of the mentioned field is empty or is invalid, then display the error and go to 3. 6. if the blockchain transaction is failed (not enough gas, network error) , then display the message "Transaction Failed" and go to 4. 7. if the product creation event is not confirmed with the timeout period, then display the message "Confirmation Timeout " and the suggestion is to check the status of transaction.

Use Case 3. Mint NFT

Flow of activities for scenario of Mint NFT

Main Flow:

1. When product was created successfully, the system calls the mint() function on the ProductNFT contract via supply chain automatically.
2. The smart contract generates a unique Token ID and creates the NFT.
3. The system links the Product ID to the Token ID in the mappings.
4. The system displays "NFT Minted Successfully" with Token ID and transaction hash.

Exception Conditions: 1. if blockchain transaction fails (insufficient gas) then show "Transaction Failed" and go to use case 2. 2. if nft minting event is not confirmed then show "Minting Failed" and suggest to check transaction status.

Use Case 4. Transfer Product

Flow of activities for scenario of Transfer Product

Main Flow:

1. The holder selects "Transfer Product" button from the main menu.
2. The system displays the "Transfer Product" interface.
3. The holder enters the Product ID.
4. The system verifies the Product ID exists and retrieves current product information.
5. The system displays the product details and current holder information.
6. The holder enters the recipient's wallet address.
7. The system validates the recipient is an authorized participant.
8. The system validates the transfer is allowed based on current status and participant types.
9. The holder enters the transfer location.
10. The holder clicks "Submit" to transfer the product.
11. The system calls the transfer_product() smart contract function.
12. The smart contract updates the current holder to the recipient address.
13. The system adds a trace record: "Product Transferred" with from/to addresses.
14. The system displays "Transfer Successful" with transaction confirmation.

Exception Conditions: 4. if does not exist for the Product ID, then show Product Not Found and go to use case2. 7. if is not of an authorized participant as recipient, then show Recipient Not Authorized and go to step 5. 8. if the transfer is not allowed at status rules (e.g. transfer while InTransit), then show Transfer Not Allowed at Current Status and go to step 3. 10. if is not the current holder of the product, then show Only current holder can transfer and go to step 3. 11. if the blockchain transaction does not succeed, then show Transaction Failed and go to step 10.

Use Case 5. Update Product Status**Flow of activities for scenario of Update Product Status**

Main Flow:

1. When ownership was transferred to current holder from the last one, the participant selects "Update Status" button from the main menu.
2. The system displays the "Update Product Status" interface.
3. The participant enters the Product ID.
4. The system verifies the Product ID exists and retrieves current product information.
5. The system displays the product details and current status.
6. The participant selects the new status from the available options based on their role.
7. The participant enters location and optional metadata.
8. The participant clicks "Submit" to update the status.
9. The system calls the `update_status()` smart contract function.
10. The smart contract validates the status transition is allowed for the participant's role.
11. The system adds a new trace record with timestamp, participant, action, and location.
12. The system displays "Status Updated Successfully" with the new status confirmation.

Exception Conditions: 4. If the Product ID does not exist, then system shows "Product Not Found" and goes to step 3. 6. If the participant try to set an invalid status transition (e.g., what is trying to set as Sold by Manufacturer), then system shows "Invalid Status Transition" and goes to step 6. 8. If the participant is not the current holder of the product, then system shows "Only current holder can update status" and goes to step 3. 9. If the blockchain transaction fails, then system shows "Transaction Failed" and goes to step 8.

Use Case 6. Recall Product

Flow of activities for scenario of Recall Product

Main Flow:

- 1.The Brand selects "Recall Product" button from the main menu.
- 2.The system displays the "Product Recall" interface.
- 3.The Brand enters the Product ID to recall.
- 4.The system verifies the Product ID exists and retrieves current product information.
- 5.The system displays the product details and current status.
- 6.The Brand enters the recall reason in the provided text field.
- 7.The Brand clicks "Submit" to recall the product.
- 8.The system calls the recall_product() smart contract function.
- 9.The smart contract validates the caller is the Brand.
- 10.The smart contract updates the product status to "Recalled".
- 11.The system adds a trace record: "Product Recalled" with the reason.
- 12.The system emits the StatusUpdated event with Recalled status.
- 13.The system displays "Product Recalled Successfully" with confirmation.

Exception Conditions: 4. Product ID not exist : Product Not Found; then return to step 3. 6. Recall reason is empty: Please enter recall reason; then return to step 6. 8. product has already recalled : Product Already Recalled; then return to step 3. 9. blockchain transaction fail : Transaction Failed; then return to step 7.

3. Market & Partners

3.1 Market Analysis

Global supply chain management software's market is gradually growing, luxury industry's growth rate is significantly higher than the average level. According to a new report by MarketsandMarkets, The global Supply Chain Management Market is expected to expand from USD 38.51 billion in 2025 to USD 58.42 billion by 2030, representing a Compound Annual Growth Rate (CAGR) of 8.7%[1]. When it comes to blockchain part, the market size is valued at USD 3.96 Billion in 2025E and is projected to reach USD 95.52 Billion by 2033, growing at a CAGR of 48.88% during 2026–2033, based on the S&S Insider data[2]. Focused on the luxury area, we decide to consider the jewelry market as the representative of the luxury market. According to the Market Reports world data, the global Blockchain In The Jewelry Supply Chain Market size was valued approximately USD 0.42 Billion in 2025 and will touch USD 1.34 billion by 2034, growing at a compound annual growth rate (CAGR) of 12.30% from 2025 to 2034[3]. Based on the data listed above, we assume that the penetration rate of blockchain-based of supply chain Management software is approximately around 10.3%. In addition, This share is expected to continue to rise as the global luxury industry accelerates the adoption of technologies such as anti-counterfeiting traceability and digital product passports. Luxury supply chain management has long faced two core pain points. Firstly, the supply chain transparency is seriously insufficient. The luxury supply chain involves multiple links from raw material procurement, production and processing, logistics and transportation to terminal retail, and the information of each link is scattered in different databases and management systems. Moreover more, The problem of counterfeit goods continues to erode brand value and consumer trust. According to a 2025 report by the OECD and the EUIPO, global trade in counterfeit goods reached approximately USD 467 billion in 2021, equivalent to 2.3% of total world imports. While this absolute value increased from USD 464 billion in 2019, its relative share of global trade declined from 2.5% over the same period[4]. At the same time, the EU's Sustainable Product Ecodesign Regulation (ESPR) has officially entered into force in 2024, requiring DPP requirements for nearly 30 product categories to be covered by 2030, further exacerbating the

compliance pressure of traditional supply chain management systems[5]. In the future, the growth potential of the blockchain supply chain management software market in the luxury sector mainly comes from three core drivers. First, the rigid deployment needs brought about by regulatory compliance. The EU Digital Product Passport (DPP) will be enforced in phases from 2027[6], and blockchain has become the preferred underlying technology for realizing product life cycle traceability with its immutable and full-link traceability. For example, Leading luxury groups including LVMH and Prada have jointly established the Aura Blockchain Consortium to trace the lifecycle of their products and access information about the design, raw materials, manufacturing and distribution. Second, technological progress and cost optimization open up space for large-scale popularization. The functional maturity of BaaS as a full-scale enterprise enabler and the availability of cost-effective cloud environments have greatly simplified blockchain network deployment and rapid scaling for diverse modern organizations.[7]. Up to now, some of the world's leading luxury brands have actively explored blockchain traceability solutions, and industry applications have completed early verification. To summarize, the luxury blockchain supply chain management software market is accelerating its transition from the early pilot stage of leading brands to the large-scale landing stage of the whole industry.

3.2 Potential Partners

The successful implementation of this system requires close collaboration with core stakeholders across the luxury supply chain. Our potential cooperation partners are divided into four core types, with corresponding collaboration values as follows:

Core Types	Partners	Collaboration Value
Brand	1.LVMH, Coach, Rolex 2.independent haute couture designer brands	1. Top brands deliver industry benchmark effect to rapidly build market trust 2. Solve the core anti-counterfeiting pain points of brands and enhance product premium capacity 3. Meet the requirements of global compliance policies such as the EU Digital Product Passport (DPP)

Manufacturer	1. manufacturing factories specializing in luxury leather goods, jewelry and watches 2. ESG-compliant raw material suppliers 3. Traditional master craft studios	1. End-to-end documentation of production processes and raw material sources to consolidate the foundation of product traceability 2. Realize production capacity visualization and optimize the production planning of brand owners 3. Establish a manufacturer credit system to promote precise matching between high-quality production capacity and brand owners
Logistic	Professional luxury goods logistics providers: DHL Luxury Goods Logistics, FedEx Premium Jewelry Shipping Service	1. Realize real-time tracking of product transportation status and mitigate the risk of product tampering 2. Upload digital signatures of logistics nodes to the blockchain to enhance the credibility of traceability 3. Optimize delivery routes and warehousing layout to improve overall supply chain efficiency
Retailer	department stores, global directly operated stores (DOS) of luxury brands, and vertical luxury e-commerce platforms such as Farfetch	Collect terminal consumption data to reverse-optimize supply chain decisions for brand owners

4. Competition & Risks

4.1 Competitive Analysis

The luxury supply chain and anti-counterfeiting market are evolving rapidly. The market we propose for decentralized applications (DApps) includes both traditional enterprise software and emerging Web3 solutions.

4.1.1 Traditional Supply Chain Management System

Competitors: Oracle SCM, SAP, IBM Sterling

Analysis: They are traditional system solutions providers that rely on centralized databases. While they are mature and efficient, they lack transparency to end consumers and are vulnerable to internal data tampering.

Our advantage is that, with our blockchain-based solution, we offer "trustless transparency". If we use a distributed ledger, all the stakeholders, including the consumer, can see the origin of the luxury good, and he does not need to trust one centralized authority blindly.

4.1.2 Exist blockchain competitors

Competitors: VeChain, Aura Blockchain Alliance

Analysis: These are strong competitors on the luxury sector. Nevertheless, these are systems that are closed, very focused on business users and hard to integrate for an independent or mid-range luxury brand.

Our advantage: Our dual-contract architecture is lightweight, highly decoupled and easy to deploy. We bridge B2B supply chain tracking (manufacturer -> logistics -> retailers) with B2C interactions: we automatically mint ERC-721 NFTs as a certificate of digital ownership of an end consumer.

4.2 Risks and Mitigation

As we mentioned, although our technology design is quite robust there are some risks regarding the implement of the block chain solutions in the physical luxury market. We have found these risks and some ways to deal with them.

4.2.1 Reality and digital binding risks

Our blockchain enforces the immutability of the digital records, but we cannot prevent the bad actors to replace the genuine physical luxury goods with fake goods when they transfer the genuine NFTs.

Therefore, we may need to integrate the hardware-based security technology at the same time. For instance, embed the encrypted tags / tamper-proof QR code into physical luxury items. The physical tag will be cryptographically tied to the blockchain product ID and unique NFT token ID, so that the physical item and digital record are bound together.

4.2.2 Centralization and key management risks

In our current model, the brand side (contract owner) has the authority to authorize the nodes (manufacturers, logistics, retailers) and to recall the products. So, in our model, the brand side has the highest authority. If the private key of the brand (say, the MetaMask account) is cracked, the integrity of the whole is in danger.

Therefore, we intend to provide brands with wallets that implement multi-sig in the future. This eliminates a single point of failure by requiring cryptographic signatures from multiple senior management personnel for critical operations.

4.2.3 User adoption and user experience

Supply chain operations personnel (factory workers, truck drivers) and retail consumers are often unfamiliar with Web3 tools. Requiring users to install MetaMask, manage seed phrases, and pay cryptocurrency "gas fees" for each status update (in transit, shipped) constitutes a significant barrier to entry.

For customer convenience, we have developed a user-friendly frontend that simplifies interaction. And in future iterations, we will implement Account Abstraction and Meta-Transactions. This will allow brands to pay gas fees on behalf of their supply chain partners, providing a seamless "Web 2.0-style" experience where participants can update their status with one single click, eliminating the need for cryptocurrency balances.

5. Value & Distribution

5.1 Value Proposition

Our system brings value to consumers, businesses and the whole luxury industry.

5.1.1 What Consumers Gain

The greatest anxiety of the general public about shopping for expensive goods in terms of authenticity. Proof that can answer this problem in our system. After scanning the Products' QR Codes with Block Chain technology to identify and ensure the truthfulness of them. No waiting period before the expert opinion. No photo upload to the authentication service. Just a click away for them on their phones.

Beyond just Yes or No responses, we show the whole process. Every stage, from the manufacturing date and place to customs clearance, warehousing arrival and Store Delivery Time Stamped With Timestamps and Authorised Participants have Signed [8]. The ordinary certificate confirms the legitimacy of the goods. Step-by-step, our system will tell you that it is an authentic one.

And because blockchain records cannot be altered, this verification never expires. Whether kept by oneself over many years or sold later, the proof will remain attached to it. This peace of mind cannot be obtained by any kind of document.

5.1.2 Which Businesses Benefit

In terms of benefits for the brand, these are as follows:

First, Brand Protection. Counterfeits are immediately recognisable as such due to their missing chain-verification trail. Each real product has a permanent trace that counterfeit goods cannot match.

Secondly, Supply chain Visibility. Traditional Systems do not provide an integrated view of all participants; neither manufacturers nor retailers have a comprehensive understanding of the situation during product transportation or arrival time. The System Can Close This Gap. All participants see the entire chain and make data-driven decisions that avoid guesswork.

Thirdly, Customers' Trust may result in competition. brands that have blockchain certification lead in terms of this area. Therefore, more consumers will be drawn to verified products. Finally, auditing is oversimplifying it. All status changes, all transfers, all actions of participants are stamped with timestamps on-chain automatically. The answers have been prepared beforehand; when asked, they will come up. No more Weeks of collecting data and monitoring Emails [9].

5.1.3 What the Industry Gets

If sufficient brands adopt a common standard across the whole luxury system. The traditional method of verification allows users to confirm that the goods come from several different sources at once. Secondhand Markets' confidence in selling their goods. Fraud does not happen gradually at all stages of the supply chain due to the lack of traceability after introducing a false item.

5.2 Business Model and Distribution

5.2.1 Business Model

We will generate income from three sources, all are in line with what our contributions can achieve:

1. Platform service fee: between 1%-2% of the products added or transactions conducted by client brands.
2. NFT minting fees: for creating new product NFTs, the fee will be relatively low, including the cost of gas and a guaranteed profit component.
3. Premium analytical services: for enterprise users, basic functionality is available to smaller enterprises at a low price, while advanced data analytics are provided as paid premium services.

Our simplified Price Policy is: Sell by the Value of Things, Not Technology. If we do not save the brand any money or reduce its risks, then we cannot charge.

5.2.2 Market Introduction and Distribution Plan

We will promote the system in three phases, with clear focus and targets for each stage:

Period	Focus	Target
Months 1-3	MVP + Pilot	2-3 brands, 500-1000 products
Months 4-6	Early Growth	10-15 brands, logistics partners
Months 7-12	Expansion	50+ brands, API launch

Specifically, we will first launch through a restricted number-of-participants trial for two to three brand sponsors. Proving that the system can work well under actual circumstances and collecting cases of application. With satisfactory outcomes, we will add 10-15 more brands; carry out cases studies; cooperate with industry media; and approach logistics partner businesses that can introduce multiple products within the network. At last, Open up our platform's public API to integrate it with secondary markets and strive for industry acceptance as an indication of luxurious origins. It does more than track this object. A type of guarantee based solely on mutual confidence among parties involved in a transaction.

6. Conclusion

This project uses blockchain, smart contract and NFT technology to build a luxury supply chain traceability system. It solves the main problems of traditional luxury supply chains, like lack of transparency, easy tampering of centralized data, low efficiency of ownership transfer, and counterfeit goods. The system uses two smart contracts to work together, and records the whole lifecycle of luxury products on the blockchain. It can help consumers check product authenticity, help brands manage supply chain and prevent counterfeits, and also help all participants in the supply chain share information. We also analyzed the market, competitors, and potential risks of the system, and made corresponding solutions for the risks. This system provides a basic solution for luxury supply chain tracking and product ownership verification.

7. References

- [1] MarketsandMarkets. “Supply Chain Management (SCM) Market Worth \$58.42 Billion by 2030- Exclusive Report by MarketsandMarketsTM.” Prnewswire.com, Cision PR Newswire, 8 July 2025, www.prnewswire.com/news-releases/supply-chain-management-scm-market-worth-58-42-billion-by-2030--exclusive-report-by-marketsandmarkets-302499834.html.
- [2] S&S Insider. “Blockchain in Supply Chain Market to Hit USD 95.52 Billion by 2033, Driven by Rising Demand for Transparency and Traceability | SNS Insider | Valuespectrum.com.” Valuespectrum.com, 2025, www.valuespectrum.com/corporate_news/1355358-blockchain-in-supply-chain-market-to-hit-usd-95-52-billion-by-2033-driven-by-rising-demand-for-transparency-and-traceability-sns-insider.
- [3] Market Reports world. “Blockchain in the Jewelry Supply Chain Market Size and Trends Research [2034].” Marketreportsworld.com, 2026, www.marketreportsworld.com/market-reports/blockchain-in-the-jewelry-supply-chain-market-14714107.
- [4] Fraser, David. “The Impact of Counterfeit Goods in Global Commerce - Ocean Tomo.” Ocean Tomo, 3 June 2025, ocean_tomo.com/insights/the-impact-of-counterfeit-goods-in-global-commerce/.
- [5] European Parliament and the Council of the European Union. “Regulation (EU) 2024/1781 of the European Parliament and of the Council of 13 June 2024 establishing a framework for the setting of ecodesign requirements for sustainable products, amending Directive (EU) 2020/1828 and Regulation (EU) 2023/1542 and repealing Directive 2009/125/EC.” EUR-Lex, 2024, <http://data.europa.eu/eli/reg/2024/1781/oj>.
- [6] trace for good. “Digital Product Passport | Trace for Good.” Traceforgood.com, 2026, www.traceforgood.com/solutions/regulation/dpp.
- [7] Taylor, Kari. “Blockchain-As-a-Service in 2025: A New Frontier for Development Services.” Coruzant Technologies, 20 Oct. 2025, coruzant.com/blockchain/blockchain-as-a-service-in-2025-a-new-frontier-for-development-services. Accessed 4 Apr. 2026.
- [8] Jang, J., & Kang, J. Blockchain meets luxury: The role of NFT authentication in luxury retail platforms. Journal of Retailing and Consumer Services, 2025 <https://ideas.repec.org/a/eee/joreco/v84y2025ics0969698925000414.html>

- [9] Ballini, E., Battaglia, L., & Cedrola, E. Blockchain in luxury fashion: a collaborative approach to innovation and trust. *Marketing Trends Congress Proceedings*, 2025
<https://archives.marketing-trends-congress.com/2025/pages/PDF/095.pdf>
- [10] Azimi, S., Goltari, A., Ivaki, N., & Laranjeiro, N. (2025). A systematic review on smart contracts security design patterns. *Empirical Software Engineering*, 30(4).
<https://doi.org/10.1007/s10664-025-10646-w>
- [11] Rahman, J., Rahman, H., Islam, N., Tanchangya, T., Ridwan, M., & Ali, M. (2025). Regulatory Landscape of Blockchain Assets: Analyzing the Drivers of NFT and Cryptocurrency Regulation. *BenchCouncil Transactions on Benchmarks, Standards and Evaluations*, 100214. <https://doi.org/10.1016/j.tbench.2025.100214>

Appendix I: Group Project source code

supply_chain_smart.sol:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.30;

/*
 * Luxury Supply Chain Smart Contract
 * Records the entire lifecycle of luxury goods from production to sale
 * Group: Group 30
 */

contract luxury_supply_chain{

// State variables
address public owner;
uint256 public Productcounter;
mapping (uint256=>Product) public products;
mapping (uint256=>Trace_record[]) public producttrace;
mapping (address=>bool) public authorizedParticipant;
mapping (address=> Participation_type) public participantTypes;

// Participants enum
enum Participation_type{
    None,           //0 - Unauthentic
    brand,          //1 - Brand like LV, Rolex
    Manufacturer,   //2 - Factory
    Logistics,      //3 - Logistics
    Retailer        //4 - Retailer
}

// Product status enum
enum product_status{
    Created,        //0
    InProduction,   //1
    Produced,       //2
}
```

```

    InTransit,      //3 - Transporting
    Shipped,        //4 - Shipped for delivery
    Instore,        //5
    sold,           //6
    Recalled         //7
}

```

// Product struct

```

struct Product {
    uint256 id;
    string name;
    string brand;
    string sku;
    string description;
    string imageUrl;
    uint256 createdAt;
    uint256 updatedAt;
    address currentHolder;
    product_status status;
    bool is_existed;
}

```

// Trace record struct

```

struct Trace_record{
    uint256 timestamp;
    address participant;
    Participation_type participation_Type;
    string action;
    string location;
    string metadata;
}

```

// Events

```

event ParticipantsAuthorized(address indexed participant, Participation_type participation_Type);
event ParticipantRevoked(address indexed participant);
event ProductCreated(uint256 indexed productId, string name, string brand);
event StatusUpdated(uint256 indexed productId, product_status new_status, address indexed

```

```

updatedBy);
event TraceAdded(uint256 indexed productId, address indexed participant, string action);
event OwnershipTransferred(uint indexed productId,address from, address to);

// Modifiers
modifier onlyauthorized(){
    require(authorizedParticipant[msg.sender],"only authentic participant can use this function");
    _;
}
modifier productExists(uint256 _productId) {
    require(products[_productId].is_existed, "Product does not exist");
    _;
}

// Constructor - sets deployer as Brand automatically
constructor(){
    owner = msg.sender;
    authorizedParticipant[msg.sender] = true;
    participantTypes[msg.sender] = Participation_type.brand;
    Productcounter=0;
}

// Authorize a new participant (only Brand can do this)
function authorizeparticipant (address _participant, Participation_type _type )public
onlyauthorized {
    require(_participant != address(0), "Invalid address!!");
    require(participantTypes[msg.sender] == Participation_type.brand,"Only brand can authorize
participants");
    require( _type == Participation_type.Manufacturer || _type == Participation_type.Logistics ||
_type == Participation_type.Retailer, "Brand can only authorize manufacturer/logistics/retailer");
    authorizedParticipant[_participant]=true;
    participantTypes[_participant]=_type;
    emit ParticipantsAuthorized(_participant, _type);
}

// Revoke a participant's authorization (only Brand can do this)
function revokeParticipant(address _participant) public onlyauthorized {

```

```

        require(authorizedParticipant[_participant], "Participant not authorized");
        require(participantTypes[msg.sender] == Participation_type.brand, "Only brand can revoke
participants");
        authorizedParticipant[_participant]=false;
        participantTypes[_participant]=Participation_type.None;
        emit ParticipantRevoked(_participant);
    }

```

// Create a new product (only Brand can do this)

```

function createproduct(
    string memory _name,
    string memory _brand,
    string memory _sku,
    string memory _description,
    string memory _imageUrl
) public onlyauthorized returns (uint256){
    require(participantTypes[msg.sender] == Participation_type.brand, "Only brand can create
products");

```

```

    Productcounter++;

```

```

    Product memory newProduct = Product({
        id: Productcounter,
        name: _name,
        brand: _brand,
        sku: _sku,
        description: _description,
        imageUrl: _imageUrl,
        createdAt: block.timestamp,
        updatedAt: block.timestamp,
        currentholder: msg.sender,
        status: product_status.Created,
        is_existed: true
    });

```

```

    products[Productcounter] = newProduct;

```

```

    _addTracerecord(
        Productcounter,
        "Product Created",
        "Brand Headquarters",
        ""
    );

    emit ProductCreated(Productcounter, _name, _brand);
    return Productcounter;
}

// Update product status (only current holder can update)
function update_status(
    uint256 _productid,
    product_status new_status,
    string memory _location,
    string memory _metadata
) public onlyauthorized productExists(_productid) {
    Product storage product = products[_productid];

    require(
        product.currentholder == msg.sender,
        "Only product owner can update status"
    );

    if (product.status == product_status.sold) {
        require(
            new_status == product_status.Recalled && participantTypes[msg.sender] ==
Participation_type.brand,
            "Only brand can recall sold products"
        );
    }

    if (product.status == product_status.sold && new_status == product_status.Instore) {
        require(
            false,
            "Cannot change sold product back to instore"
        );
    }
}

```



```

    );
}

product.status = new_status;
product.updatedAt = block.timestamp;

string memory action = _getStatusaction(new_status);
_addTracerecord(_productid, action, _location, _metadata);

emit StatusUpdated(_productid, new_status, msg.sender);
}

// Transfer product ownership
function transfer_product(
    uint256 _productid,
    address _to,
    string memory _location
) public onlyauthorized productExists(_productid) {
    require(_to != address(0), "Invalid address");
    require(authorizedParticipant[_to], "Recipient is not authorized");

    Product storage product = products[_productid];
    address from = product.currentholder;

    product.currentholder = _to;
    product.updatedAt = block.timestamp;
    _addTracerecord(_productid, "Product Transferred", _location, "");

    emit OwnershipTransferred(_productid, from, _to);
}

// Recall a product (only Brand can do this)
function recall_product(
    uint256 _productid,
    string memory _reason
) public onlyauthorized productExists(_productid) {
    require(

```

```

        participantTypes[msg.sender] == Participation_type.brand,
        "Only brand can recall products"
    );

    Product storage product = products[_productid];
    product.status = product_status.Recalled;
    product.updatedAt = block.timestamp;

    _addTracerecord(_productid, "Product Recalled", "", _reason);

    emit StatusUpdated(_productid, product_status.Recalled, msg.sender);
}

// Get product info by ID
function getProduct(uint256 _productid) public view productExists(_productid) returns (
    uint256 id,
    string memory name,
    string memory brand,
    string memory sku,
    string memory description,
    string memory imageUrl,
    uint256 createAt,
    uint256 updatedAt,
    address currentholder,
    product_status status
){
    Product memory product = products[_productid];
    return (
        product.id,
        product.name,
        product.brand,
        product.sku,
        product.description,
        product.imageUrl,
        product.createAt,
        product.updatedAt,
        product.currentholder,

```

```

        product.status
    );
}

// Get trace record count for a product
function getTracecount(uint256 _productid) public view productExists(_productid) returns
(uint256) {
    return producttrace[_productid].length;
}

// Get a specific trace record
function getTracerecord(uint256 _productid, uint256 _index)
    public view productExists(_productid)
    returns (
        uint256 timestamp,
        address participant,
        Participation_type participation_Type,
        string memory action,
        string memory location,
        string memory metadata
    )
{
    require(_index < producttrace[_productid].length, "Index out of bounds");

    Trace_record memory record = producttrace[_productid][_index];
    return (
        record.timestamp,
        record.participant,
        record.participation_Type,
        record.action,
        record.location,
        record.metadata
    );
}

// Get all trace records for a product
function getAllTracerecords(uint256 _productid) public view productExists(_productid) returns

```

```

(Trace_record[] memory)
{
    return producttrace[_productid];
}

// Batch get status of multiple products
function getBatchstatus(uint256[] memory _productids) public view returns (product_status[]
memory statuses)
{
    uint256 length = _productids.length;
    statuses = new product_status[](length);

    for (uint256 i = 0; i < length; i++) {
        require(products[_productids[i]].is_existed, "Product does not exist");
        statuses[i] = products[_productids[i]].status;
    }
}

// Internal: Add trace record
function _addTracerecord(
    uint256 _productid,
    string memory _action,
    string memory _location,
    string memory _metadata
)internal {
    Trace_record memory newRecord=Trace_record({
        timestamp: block.timestamp,
        participant: msg.sender,
        participation_Type: participantTypes[msg.sender],
        action: _action,
        location: _location,
        metadata: _metadata
    });
    producttrace[_productid].push(newRecord);
    emit TraceAdded(_productid, msg.sender, _action);
}

```

```
// Internal: Get status action text
function _getStatusaction(product_status _status) internal pure returns (string memory){
    if (_status == product_status.InProduction) return "Production Started";
    if (_status == product_status.Produced) return "Production Completed";
    if (_status == product_status.InTransit) return "In Transit";
    if (_status == product_status.Shipped) return "Shipped for Delivery";
    if (_status == product_status.Instore) return "Arrived at Store";
    if (_status == product_status.sold) return "Product Sold";
    if (_status == product_status.Recalled) return "Product Recalled";
    return "Status Updated";
}
}
```

NFT.sol:

```
// SPDX-License-Identifier: MIT
```

```
pragma solidity ^0.8.30;
```

```
// NFT contract for issuing unique certificates per productId
```

```
interface IERC721 {
```

```
    event Transfer(address indexed from, address indexed to, uint256 indexed tokenId);
```

```
    event Approval(address indexed owner, address approved, uint256 tokenId);
```

```
    function balanceOf(address owner) external view returns (uint256 balance);
```

```
    function ownerOf(uint256 tokenId) external view returns (address owner);
```

```
    function safeTransferFrom(address from, address to, uint256 tokenId) external;
```

```
    function transferFrom(address from, address to, uint256 tokenId) external;
```

```
    function approve(address to, uint256 tokenId) external;
```

```
    function getApproved(uint256 tokenId) external view returns (address operator);
```

```
}
```

```
contract ProductNFT {
```

```
    string public name = "LuxuryProductNFT";
```

```
    string public symbol = "LPNFT";
```

```
    address public owner;
```

```
    mapping(uint256 => address) private _owners;
```

```
    mapping(address => uint256) private _balances;
```

```
    mapping(uint256 => address) private _tokenApprovals;
```

```

mapping(uint256 => uint256) public productToNFT;
mapping(uint256 => uint256) public NFTToProduct;
mapping(uint256 => string) private _tokenURIs;
uint256 private _tokenIdCounter;
address public supplyChainContract;

event Transfer(address indexed from, address indexed to, uint256 indexed tokenId);
event Approval(address indexed owner, address approved, uint256 tokenId);
event NFTMinted(uint256 indexed tokenId, uint256 indexed productId, address to);
event ContractAddressUpdated(address newContract);

modifier onlyOwner() {
    require(msg.sender == owner, "Only owner can call this function");
    _;
}

modifier onlySupplyChain() {
    require(msg.sender == supplyChainContract, "Only supply chain contract can call this");
    _;
}

modifier exists(uint256 tokenId) {
    require(_owners[tokenId] != address(0), "Token does not exist");
    _;
}

constructor() {
    owner = msg.sender;

```

```

        _tokenIdCounter = 0;
    }

    // this func is used for updating the address of supplychain smart contract
    function setSupplychain_contract(address _contract) public onlyOwner {
        supplyChainContract = _contract;
        emit ContractAddressUpdated(_contract);
    }

    //build NFT

    function mint(
        address _to,
        uint256 _productId,
        string memory _tokenURI//the note about a NFT
    ) public onlySupplyChain returns (uint256) {
        return _mintInternal(_to, _productId, _tokenURI);
    }

    function mintByOwner(
        address _to,
        uint256 _productId,
        string memory _tokenURI
    ) public onlyOwner returns (uint256) {
        return _mintInternal(_to, _productId, _tokenURI);
    }

    //internal function, core logic of build a nft

    function _mintInternal(
        address _to,
        uint256 _productId,
        string memory _tokenURI

```



```

) internal returns (uint256) {

    require(_to != address(0), "Invalid address");

    require(productToNFT[_productId] == 0, "Product already has NFT");

    _tokenIdCounter++;

    uint256 newTokenId = _tokenIdCounter;

    _owners[newTokenId] = _to;

    _balances[_to] += 1;

    _tokenURIs[newTokenId] = _tokenURI;

    productToNFT[_productId] = newTokenId; //product id--->nft id
    NFTToProduct[newTokenId] = _productId; // vise versa

    emit Transfer(address(0), _to, newTokenId);

    emit NFTMinted(newTokenId, _productId, _to);

    return newTokenId;
}

```

```

function transferFrom(

    address _from,

    address _to,

    uint256 _tokenId

) public exists(_tokenId) {

    address tokenOwner = _owners[_tokenId];

    require(tokenOwner == _from, "From address is not the owner");

    require(

```

```

        msg.sender == _from || msg.sender == _tokenApprovals[_tokenId], "Not authorized to
transfer"

    );

    require(_to != address(0), "Invalid to address");

    if (msg.sender != _from && _tokenApprovals[_tokenId] != msg.sender) {
        revert("Not authorized to transfer");
    }

    _tokenApprovals[_tokenId] = address(0);
    emit Approval(_from, address(0), _tokenId);

    _balances[_from] -= 1;
    _balances[_to] += 1;

    _owners[_tokenId] = _to;

    emit Transfer(_from, _to, _tokenId);
}

function safeTransferFrom(
    address _from,
    address _to,
    uint256 _tokenId
) public exists(_tokenId) {
    transferFrom(_from, _to, _tokenId);
}

```

```

function safeTransferFrom(
    address _from,
    address _to,
    uint256 _tokenId,
    bytes memory data
) public exists(_tokenId) {
    transferFrom(_from, _to, _tokenId);
}

// to approve a address to do something on its nft

function approve(address _to, uint256 _tokenId) public exists(_tokenId) {
    address tokenOwner = _owners[_tokenId];
    require(msg.sender == tokenOwner, "Not token owner");

    _tokenApprovals[_tokenId] = _to;
    emit Approval(tokenOwner, _to, _tokenId);
}

function ownerOf(uint256 _tokenId) public view returns (address) {
    address tokenOwner = _owners[_tokenId];
    require(tokenOwner != address(0), "Token does not exist");
    return tokenOwner;
}

function balanceOf(address _owner) public view returns (uint256) {
    require(_owner != address(0), "Invalid address");
    return _balances[_owner];
}

```

```

function getApproved(uint256 _tokenId) public view exists(_tokenId) returns (address) {
    return _tokenApprovals[_tokenId];
}

function tokenURI(uint256 _tokenId) public view exists(_tokenId) returns (string memory) {
    return _tokenURIs[_tokenId];
}

function totalSupply() public view returns (uint256) {
    return _tokenIdCounter;
}

function getNFTByProduct(uint256 _productId) public view returns (uint256) {
    require(productToNFT[_productId] != 0, "Product has no NFT");
    return productToNFT[_productId];
}

function getProductByNFT(uint256 _tokenId) public view exists(_tokenId) returns (uint256) {
    return NFTToProduct[_tokenId];
}
}

```

Appendix II: Group Project Test screenshot

```
> supply-chain-nft@1.0.0 test
> hardhat test

Supply Chain Unit Tests
  Deployment
    ✓ Should set the right owner
    ✓ Should set owner as an authorized brand
  Authorization
    ✓ Should allow brand to authorize a manufacturer
    ✓ Should not allow unauthorized users to authorize others
    ✓ Should allow brand to revoke a participant
  Product Management
    ✓ Should allow brand to create a product
    ✓ Should update product status correctly
    ✓ Should allow transfer of ownership
    ✓ Should allow brand to recall a product

NFT Unit Tests
  Deployment
    ✓ Should set the correct name and symbol
    ✓ Should set the deployer as owner
  Configuration
    ✓ Should allow owner to set supply chain contract address
    ✓ Should not allow non-owner to set supply chain contract
  Minting
    ✓ Should allow owner to mint by owner
    ✓ Should allow supply chain contract to mint
    ✓ Should revert if product already has an NFT
  Transfers and Approvals
    ✓ Should allow token owner to transfer
    ✓ Should allow approved address to transfer
    ✓ Should revert if non-owner tries to transfer

Supply Chain and NFT Integration Tests
  ✓ Should complete a full product lifecycle successfully (82ms)

Performance & Gas Usage Tests
  Gas used for createproduct: 376346
  ✓ Should measure gas for product creation
  Gas used for update_status: 161069
  ✓ Should measure gas for status update
  Gas used for transfer_product: 141960
  ✓ Should measure gas for transfer_product
  ✓ Should measure gas for batchGetStatus
```